Victim–Attacker Interaction Duration Analysis

Task 2 – Complete Implementation Specification for Code Generation

1. Objective

This module measures how long an attacker and victim stay in close interaction before, during, and after a suspected chain-snatching event, then classifies the interaction as **suspicious** or **normal** based on duration and motion patterns.

The module must:

- Work on tracked person IDs produced by YOLOv8 + DeepSORT/ByteTrack.
- Maintain a **state machine per (attacker_id, victim_id) pair** over time.
- Compute start and end times, total duration, and optional pre/post-contact durations.
- Use **distance thresholds** and **camera-agnostic motion (bhs: bounding-box heights per second)** to flag very short, sudden, high-motion interactions as suspicious.
- Integrate cleanly with Phase 2 Social Force Model outputs (`forceevents.json`) as an event-driven mode.
- Achieve **≥ 90% accuracy** in classifying interactions at both the frame level and interaction (video) level on a labeled validation set.

2. Background and Context in Overall System

This module is Task 2 in a larger CCTV chain-snatching detection system. The broader pipeline is structured as:

- **Phase 1** — YOLOv8 person detection + DeepSORT/ByteTrack tracking.
- **Phase 2** — Social Force Model and kinematic anomaly detection, producing `forceevents.json`.
- **Phase 3** — Semantic verification using RTMPose, Mask R-CNN, and a contact detection network.

- **Immediate Evidence Capture** — Rolling frame buffer that saves video clips and keyframes around confirmed events.

Task 2 (this module) sits between tracking/force events and evidence capture:

1. Receives per-frame detections and tracks (and optionally Phase 2 force events).
2. Computes interaction durations and motion features for each attacker–victim pair.
3. Classifies each interaction as suspicious or normal.
4. Outputs a CSV report and annotated video.
5. Triggers Immediate Evidence Capture for suspicious interactions.

Why Interaction Duration Matters

Chain snatching typically completes within **0.5–2.5 seconds** of close contact. Normal conversations, handshakes, or casual encounters have much longer close-proximity windows (often > 10 seconds) or very low relative motion. By measuring duration AND motion speed, the system can separate genuine criminal interactions from false positives without needing to run expensive deep learning models on every frame.

3. Inputs

3.1 Core Inputs (per Task 2 PDF)

Input	Type	Description
`video_frames`	List of frames	Raw CCTV frames or decoded video frames
`person_detections`	List per frame	YOLOv8 detections: `bbox=(x1,y1,x2,y2)`, `confidence`, `class_id`
`tracks`	List per frame	Tracker output: `track_id`, `bbox`, `timestamp` or `frame_idx`
`attacker_ids`	List of int	Candidate attacker track IDs from upstream modules
`fps`	float	Video frame rate (e.g., 25.0 fps)

3.2 Optional Inputs

Input	Type	Description

forceevents.json	JSON file	Phase 2 Social Force Model output with flagged frame/pair/force data
pose_keypoints	Dict per frame per track	RTMPose 2D keypoints for shoulder, neck, wrist landmarks

4. Outputs

4.1 CSV Report

One row per finalized attacker–victim interaction. Columns:

Column	Type	Description
interaction_id	int	Unique ID per attacker–victim interaction (auto-incremented)
video_id	str	Identifier for the source video or camera stream
attacker_id	int	Track ID of the attacker
victim_id	int	Track ID of the victim
start_frame	int	First frame index where close interaction began
end_frame	int	Last frame index of close interaction
start_time_sec	float	Start time in seconds from video start
end_time_sec	float	End time in seconds
duration_sec	float	end_time_sec - start_time_sec
pre_contact_duration_sec	float	Time from first co-visibility of both IDs to start of close interaction
post_contact_duration_sec	float	Time from end of close interaction until they permanently separate
min_distance_norm	float	Minimum normalized distance observed during interaction
avg_distance_norm	float	Average normalized distance across interaction frames
peak_relative_speed_bhs	float	Peak relative speed in bhs (bounding-box heights per second)
max_force_magnitude	float	Max Phase 2 social force value for this pair (null if not available)
rule_label	str	Label from rule-based thresholds: suspicious, normal, or uncertain
ml_label	str	Label from ML classifier (if enabled)
ml_prob_suspicious	float	ML model's probability that interaction is suspicious (0–1)
label	str	Final label used by the system: suspicious, normal, or uncertain

triggered_evidence_capture	bool	Whether Evidence Capture was triggered for this interaction

4.2 Annotated Video

A video file where each frame includes:

- Bounding boxes for all tracked persons (default: green).
- For active interactions: attacker box in red, victim box in yellow, center-to-center line.
- Text overlay per active interaction: `INT#<id> [SUSPICIOUS/NORMAL] dur=2.3s`.
- When an interaction finalizes: brief label flash on the pair.

4.3 Evidence Capture Trigger Events

For each suspicious interaction, emit a trigger containing:

- `video_id`, `interaction_id`, `attacker_id`, `victim_id`.
- `capture_start_time_sec = start_time_sec - pre_margin_sec`.
- `capture_end_time_sec = end_time_sec + post_margin_sec`.
- `label`, `ml_prob_suspicious`.

4.4 Performance Metrics Report

Computed on labeled validation data:

- Frame-level accuracy, precision, recall, F1 for "suspicious" vs "normal" frames.
- Interaction-level accuracy, precision, recall, F1 for classifying each interaction.
- Target: **≥ 90% accuracy at both levels**.

5. Overall Architecture

5.1 Concept

Treat this module as a **state machine per (attacker_id, victim_id) pair** that runs over time. There is one state machine instance per unique pair encountered in the video. All state machines run in parallel across all frames.

5.2 High-Level Per-Frame Pipeline

For each frame, the following steps execute in order:

1. Run YOLOv8 on the frame → get person detections (bounding boxes and confidence scores).
2. Pass detections into DeepSORT or ByteTrack → get stable track IDs for all persons.
3. For each attacker ID present in this frame:
    - Identify candidate victim IDs (all other tracked persons in the same frame).
    - Compute distance features between attacker and each candidate victim.
    - Compute relative motion features, including bhs (bounding-box heights per second) speed.
4. For each (attacker_id, victim_id) pair:
    - Update the pair's state machine using current distance and motion values.
    - If the state machine just transitioned to IDLE from ACTIVE, finalize the interaction.
5. For each finalized interaction:
    - Compute duration, pre/post-contact durations, and all features.
    - Classify as suspicious or normal.
    - Append to `interaction_records`.
    - If suspicious, emit an Evidence Capture trigger.
6. Annotate the current frame for all currently ACTIVE or PENDING_END pairs.
7. Write annotated frame to output video.

5.3 Two Operating Modes

Mode 1 — Continuous (Tracking-Driven)

- Runs interaction logic on every frame for all attacker–victim pairs.
- Independent of Social Force Model.
- Recommended for initial development, testing, and debugging.

Mode 2 — Event-Driven (Integrated with Phase 2)

- Loads `forceevents.json` and restricts state machine instantiation to pairs flagged by Phase 2 or within ±1 second around those force events.
- Significantly reduces computation for long CCTV streams.

- Recommended for production deployment.

Both modes share identical state machine, distance computation, bhs velocity, and classification logic. Mode is controlled by a configuration flag `OPERATING_MODE` with values `"continuous"` or `"event_driven"`.

6. Step 1 — Person Detection and Tracking

6.1 YOLOv8 Person Detection

Use the Ultralytics YOLOv8 model configured to output only the **person** class (class ID 0 in COCO).

Inputs:

- Raw frame (BGR image from OpenCV).

Outputs per detection:

- `bbox = (x1, y1, x2, y2)` in pixel coordinates.
- `confidence` (float, 0–1).
- `class_id` (must equal person class).

Configuration variables:

Variable	Recommended Value	Purpose
`YOLO_CONF_THRESHOLD`	0.30	Minimum detection confidence to accept
`YOLO_IOU_THRESHOLD`	0.45	NMS IoU threshold
`YOLO_CLASSES`	[0]	Person class only
`YOLO_IMGSZ`	640	Input image size

Use the model variant fine-tuned on crowded CCTV footage from Phase 1 if available; otherwise use the standard COCO pre-trained YOLOv8n or YOLOv8s.

6.2 DeepSORT / ByteTrack Tracking

Feed filtered YOLOv8 detections (person class only) into **DeepSORT or ByteTrack**, as explicitly specified by the Task 2 PDF. The tracker assigns a stable `track_id` to each person across frames, handling brief occlusions via appearance features (DeepSORT) or dual-confidence association (ByteTrack).

Core tracker outputs per frame:

- `track_id` (int): stable identifier for each person across frames.
- `bbox` (x1, y1, x2, y2): updated bounding box from tracker.
- `frame_idx` (int) or `timestamp` (float seconds).

Configuration variables:

Variable	Recommended Value	Purpose
TRACKER_MAX_AGE	30	Frames to keep a lost track alive before deleting
TRACKER_MIN_HITS	3	Minimum frames a track must be seen before confirming
TRACKER_IOU_THRESHOLD	0.3	Minimum overlap for track association

Per-frame data structure to maintain:

- `tracked_persons[frame_idx]` = list of records, each with `track_id`, `bbox`, `timestamp`.
- A position history: `track_positions[track_id]` = deque of recent center positions, used to compute velocity.

7. Step 2 — Distance and Motion Computation

7.1 Bounding-Box-Based Normalized Distance (Baseline)

For each `(attacker_id, victim_id)` pair in a given frame:

Step A — Compute box centers:

$$c_A = \left(\frac{x1_A + x2_A}{2},\right)$$

$$c_V = \left(\frac{x1_V + x2_V}{2},\right)$$

Step B — Compute Euclidean distance in pixels:

$$d = \sqrt{(x_{A_center} - x_{V_center})^2 + (y_{A_center} - y_{V_center})^2}$$

Step C — Normalize by average box height:

$$H = \frac{(y2_A - y1_A) + (y2_V - y1_V)}{2}$$

$$d_{norm} = \frac{d}{H}$$

This normalization makes `d_norm` independent of the camera's zoom level or the persons' distance from the camera. A `d_norm` value of 1.0 means the centers are separated by exactly one average person height.

7.2 Overlap and Contact Heuristics

Compute additional geometric signals alongside `d_norm`:

Horizontal overlap ratio:

$$\text{h_overlap} = \frac{\max(0,}{\min(x2_A - x1_A,}$$

- If `h_overlap > 0.3`, persons are side-by-side; flag as additionally close.

Intersection over Union (IoU):

$$\text{IoU} = \frac{\text{area of intersection}}{\text{area of union}}$$

- If `IoU > 0.1`, persons are very close or overlapping; use as supplementary closeness signal.

These signals disambiguate crowded scenes where two people may have similar `d_norm` but very different spatial configurations.

7.3 Optional Pose-Based Distance

When RTMPose keypoints are available from the pose estimation module already running in the pipeline, compute a semantic distance:

- For attacker: use wrist keypoints (left wrist index 9, right wrist index 10 in RTMPose's COCO body schema).
- For victim: use neck/upper torso keypoint (neck midpoint from average of left shoulder index 5 and right shoulder index 6).

Compute:

$$d_{hand_neck} = \min(\|wrist_L^A - neck^V\|,)$$

Normalize by victim torso length (distance from neck midpoint to midpoint of hips). Store this as `pose_hand_neck_dist_norm`. Use it as an additional feature in the ML classifier and for debug logging. This is optional and only activates when `USE_POSE_DISTANCE = True` and keypoints are available.

8. Step 3 — bhs Velocity Computation (Gap 5)

To correctly implement "high-motion" detection regardless of camera distance and zoom, use **bhs: bounding-box heights per second**.

8.1 Definition

For a person with `track_id` across two consecutive frames at times t_1 and t_2:

$$\Delta c = \sqrt{(\Delta x_{center})^2 + (\Delta y_{center})^2}$$

$$h = y2 - y1 \text{ (box height at time } t_1\text{)}$$

$$\Delta t = t_2 - t_1 \text{ (seconds)}$$

$$\text{speed_bhs} = \frac{\Delta c}{h \cdot \Delta t}$$

A value of 1.0 bhs means the person's center moved by one full body height in one second — a jogging pace. Values above 1.5 bhs indicate fast running or lunging.

8.2 Relative Speed in bhs

For an (attacker, victim) pair:

$$\text{relative_speed_bhs} = \frac{\|\vec{v}_A - \vec{v}_V\|}{H}$$

where $\vec{v}_A$ and $\vec{v}_V$ are the 2D velocity vectors (pixels/second) of attacker and victim respectively, and H is the average box height.

8.3 Implementation Notes

- Maintain a `track_velocity[track_id]` dictionary storing the most recent velocity vector `(vx, vy)` in pixels/second.
- Compute velocity as a rolling average over the last 3–5 frames to smooth noise.

- Update `peak_relative_speed_bhs` per interaction as the running maximum.

Configuration variables for velocity:

Variable	Recommended Value	Purpose
`VELOCITY_SMOOTHING_FRAMES`	4	Frames over which to smooth velocity
`V_HIGH_BHS`	1.5	Threshold for "high-motion / sudden lunge"
`V_NOISE_FLOOR_BHS`	0.05	Speeds below this are treated as stationary

9. Step 4 — Interaction State Machine

9.1 Thresholds

Variable	Recommended Value	Purpose
`D_CLOSE`	0.7	Normalized distance threshold to consider two persons "close"
`K_START`	5	Consecutive frames below `D_CLOSE` required to confirm interaction start
`G_GAP`	8	Max gap frames above `D_CLOSE` allowed before ending interaction
`H_OVERLAP_CLOSE`	0.3	Horizontal overlap ratio supplement for closeness
`IOU_CLOSE`	0.10	IoU supplement for closeness

Calibration guidance: if the system produces too many false starts on busy footpaths, increase `K_START` to 7–8. If real snatching events are being missed because of brief attacker hesitation, increase `G_GAP` to 10–12.

9.2 State Values

Each `(attacker_id, victim_id)` pair has exactly one state at any time:

- `"IDLE"` — Not interacting. No recent close contact.
- `"PENDING_START"` — Distance is below threshold but not enough consecutive frames to confirm.
- `"ACTIVE"` — Confirmed close interaction in progress.

- `"PENDING_END"` — Distance briefly exceeded threshold; gap tolerance window open.

9.3 Per-Pair State Dictionary

Each pair stores:

```
interactions[(attacker_id, victim_id)] = {
    "state":                    "IDLE",
    "start_frame":              None,
    "start_time_sec":           None,
    "end_frame":                None,
    "end_time_sec":             None,
    "last_active_frame":        None,
    "frames_in_contact":        0,
    "frames_gap":               0,
    "min_distance_norm":        float("inf"),
    "sum_distance_norm":        0.0,
    "distance_sample_count":    0,
    "peak_relative_speed_bhs":  0.0,
    "max_force_magnitude":      0.0,
    "first_co_visible_frame":   None,
    "first_co_visible_time_sec": None,
    "last_co_visible_frame":    None,
    "last_co_visible_time_sec": None
}
```

9.4 State Transition Logic (Per Frame, Per Pair)

For each `(attacker_id, victim_id)` pair visible in the current frame:

1. Compute `d_norm`, `h_overlap`, `iou`, `relative_speed_bhs`.
2. Define `is_close = (d_norm <= D_CLOSE) OR (h_overlap > H_OVERLAP_CLOSE) OR (iou > IOU_CLOSE)`.
3. Update `first_co_visible_frame` and `last_co_visible_frame`.
4. Apply transitions:

From IDLE:

- If `is_close`: set state to `PENDING_START`, reset `frames_in_contact = 1`.
- Else: remain IDLE.

From PENDING_START:

- If `is_close`:
  - Increment `frames_in_contact`.
  - If `frames_in_contact >= K_START`: set state to `ACTIVE`, record `start_frame` and `start_time_sec`.
- Else: set state back to `IDLE`, reset `frames_in_contact = 0`.

From ACTIVE:

- If `is_close`:
  - Update `last_active_frame`, running distance and speed accumulators.
- Else:
  - Set state to `PENDING_END`, reset `frames_gap = 1`.

From PENDING_END:

- If `is_close`:
  - Set state back to `ACTIVE`. Update `last_active_frame`.
- Else:
  - Increment `frames_gap`.
  - If `frames_gap > G_GAP`:
    - Record `end_frame = last_active_frame`, `end_time_sec`.
    - Set state to `IDLE`. Reset all counters.
    - **Finalize the interaction** (proceed to duration computation and classification).

5. For any `(attacker_id, victim_id)` pair no longer visible (one or both IDs disappeared from the frame), if state is `ACTIVE` or `PENDING_END`, continue incrementing `frames_gap` as if above-threshold until `G_GAP` is exceeded, then finalize.

10. Codex Implementation Constraints (Hard Requirements)

10.1 Configuration Block

Place ALL configuration parameters in a clearly labelled CONFIG BLOCK at the top of the main script using ALL_CAPS names. Minimum required constants:

PROXIMITY_THRESH, SNATCH_MAX_DURATION_SEC, CONVO_MIN_DURATION_SEC,

FAST_APPROACH_BHS_THRESH, FAST_SEPARATION_BHS_THRESH, DIST_SMOOTH_ALPHA,

PROXIMITY_VOTE_WINDOW, PROXIMITY_VOTE_REQUIRED, TRACK_MAX_GAP_FRAMES,

SUSPICION_THRESH, W1, W2, W3, W4, MIN_INTERACTION_FRAMES, PRE_EVENT_FRAMES, POST_EVENT_FRAMES.

All duration calculations must be in seconds. Extract FPS with cap.get(cv2.CAP_PROP_FPS); if the result is 0 or None, default to 25.0.

10.2 Role Assignment Strategy

Do NOT assume victim and attacker labels are available at runtime.

Use a heuristic role-assignment strategy:

1. For each candidate pair, compute approach_velocity as the rate of distance decrease over the last N frames.

2. Compute post_proximity_separation_speed as the rate of distance increase after close contact ends.

3. The person with higher POST-separation speed is attacker.

4. The person with lower post-separation speed is victim.

5. If only two people are present in the frame, the ID with higher cumulative motion vector change may be used as attacker.

6. Store roles as (attacker_id, victim_id) per interaction event, not per frame.

10.3 Speed Normalization

Normalize all speeds to bhs (bounding-box heights per second) so the system is camera-agnostic.

Use:

bhs = pixel_displacement_per_second / mean(bbox_height_person1, bbox_height_person2)

FAST_APPROACH_BHS_THRESH = 1.5

FAST_SEPARATION_BHS_THRESH = 2.0

All motion-based heuristics must use bhs, not raw pixel speed.

10.4 Smoothing and Track Persistence

Use the following smoothing parameters:

DIST_SMOOTH_ALPHA = 0.3

PROXIMITY_VOTE_WINDOW = 8

PROXIMITY_VOTE_REQUIRED = 5

TRACK_MAX_GAP_FRAMES = 5

MIN_INTERACTION_FRAMES = 4

Use distance smoothing and voting so one-frame glitches do not create false interactions. If a track disappears briefly for fewer than TRACK_MAX_GAP_FRAMES, keep its state alive instead of resetting immediately.

10.5 Suspicion Score Formula

Use an explicit suspicion score formula:

S = W1 * (1 - norm_duration) + W2 * approach_speed_bhs + W3 * separation_speed_bhs + W4 * motion_delta

Default weights:

W1 = 0.30

W2 = 0.35

$W_3 = 0.25$

$W_4 = 0.10$

Label rule:

- label = "suspicious" if S > SUSPICION_THRESH AND duration_sec < SNATCH_MAX_DURATION_SEC

- label = "normal" if duration_sec > CONVO_MIN_DURATION_SEC

- otherwise label = "uncertain"

All weights and thresholds must be configurable at the top of the script.

10.6 Step Logging Structure

Use the same STEP-style logging structure as the Task 1 pipeline so the notebook is auditable and easy to debug:

STEP 1: Load video + initialize tracker

STEP 2: Detection + tracking loop

STEP 3: Interaction analysis (proximity, duration, scoring)

STEP 4: Role assignment (attacker/victim)

STEP 5: Metrics + evaluation, if ground truth is provided

STEP 6: Export CSV + annotated video + timeline summary

10.7 Output and Evaluation

The system must export:

- CSV report with track IDs, timestamps, distance, duration, label, and scores

- annotated output video

- simple event timeline summary

Evaluation must report:

- precision

- recall

- F1

- event-level accuracy

- duration error

Do not rely on accuracy alone.

10. Step 5 — Interaction Duration Computation

When a state machine finalizes (transitions to IDLE after ACTIVE):

Core durations:

$$\text{duration_frames} = \text{end_frame} - \text{start_frame} + 1$$

$$\text{duration_sec} = \frac{\text{duration_frames}}{\text{fps}}$$

Pre-contact duration:

$$\text{pre_contact_sec} = \text{start_time_sec} - \text{first_co_visible_time_sec}$$

This measures how long the attacker was near the victim before making contact. A very short pre-contact window (< 1 second) combined with sudden high speed is a strong suspicious signal — consistent with drive-by snatching. A long pre-contact window (> 10 seconds) may indicate loitering, which can also be suspicious but is a different pattern.

Post-contact duration:

$$\text{post_contact_sec} = \text{last_co_visible_time_sec} - \text{end_time_sec}$$

This measures how long both persons remained visible after the close contact ended. If the attacker disappears immediately after (very short post-contact), this strongly correlates with a snatch-and-flee pattern.

Average distance:

$$\text{avg_distance_norm} = \frac{\text{sum_distance_norm}}{\text{distance_sample_count}}$$

11. Step 6 — Suspicious vs Normal Classification

11.1 Rule-Based Baseline

This must always be implemented as the primary or fallback classifier.

Duration thresholds:

Variable	Recommended Value	Meaning
`T_MIN_SEC`	0.20	Interactions shorter than this are noise; label normal
`T_SHORT_SEC`	2.50	Interactions at or below this with high motion are suspicious
`T_LONG_SEC`	10.0	Interactions at or above this with low motion are normal conversations

Motion threshold:

Variable	Recommended Value	Meaning
`V_HIGH_BHS`	1.5	Relative speed in bhs above this is considered "sudden lunge"

Decision logic:

```
if duration_sec < T_MIN_SEC:
    label = "normal"   # Too short to be meaningful

elif duration_sec <= T_SHORT_SEC and peak_relative_speed_bhs >= V_HIGH_BHS:
    label = "suspicious"   # Short + sudden motion = snatch pattern

elif duration_sec >= T_LONG_SEC and peak_relative_speed_bhs < V_HIGH_BHS:
    label = "normal"   # Long + slow = conversation
```

```
elif pre_contact_duration_sec < 0.5 and peak_relative_speed_bhs >= V_HIGH_BHS:
   label = "suspicious"   # Sudden approach with no loitering

else:
   label = "uncertain"   # Default: risk-averse → treat as suspicious
                         # or "normal" if system is configured to reduce alerts
```

The `"uncertain"` fallback behavior is controlled by a configuration flag `UNCERTAIN_DEFAULT` set to either `"suspicious"` or `"normal"`.

11.2 Optional ML Classifier for ≥ 90% Accuracy Target

To achieve the target **≥ 90% accuracy** at both frame and interaction level, implement an optional lightweight ML classifier that uses the following **feature vector per finalized interaction**:

Feature	Description
`duration_sec`	Total interaction duration
`min_distance_norm`	Closest normalized distance reached
`avg_distance_norm`	Mean normalized distance during interaction
`peak_relative_speed_bhs`	Maximum relative speed in bhs
`pre_contact_duration_sec`	Time from co-visibility to first close contact
`post_contact_duration_sec`	Time from last close contact to permanent separation
`max_force_magnitude`	Max Phase 2 social force (0 if not available)
`num_gap_events`	Number of times the pair briefly exited then re-entered close range
`avg_h_overlap`	Average horizontal overlap ratio during interaction
`pose_hand_neck_dist_norm`	Optional: normalized hand-to-neck distance (if pose enabled)

Model recommendation:

- Primary: **Gradient-boosted trees (XGBoost or LightGBM)**. Handles small datasets well, is fast at inference, and produces interpretable feature importances.
- Alternative: Logistic Regression (if dataset is small, < 500 labeled interactions).
- Optional: Small MLP (2 hidden layers, 64–32 units) if dataset is larger (> 2000 interactions).

Training protocol:

- Labels: `0 = normal`, `1 = suspicious`.
- Split by **video** (not by frame or interaction): 70% train, 15% validation, 15% test. Never split the same video across train and test — this prevents information leakage.
- Do not cap the number of videos per set to any arbitrary limit (e.g., no "max 200 videos" rule). Use all available labeled data.
- Use **stratified splits** to maintain label balance.
- Apply **class-weight balancing** during training, since suspicious interactions are rarer.
- Use early stopping on validation loss with patience of 15–20 rounds; do not fix epochs at a small value like 20–25.
- Target validation accuracy of ≥ 92% to provide margin before test set evaluation.

At inference time:

- Compute rule-based label.
- Compute ML probability `p_suspicious`.
- If `p_suspicious >= 0.50`, set `ml_label = "suspicious"`.
- Final `label`: if ML is enabled and available, use `ml_label`; else fall back to `rule_label`.
- Store both `rule_label` and `ml_label` in CSV for debugging and comparison.

12. Step 7 — Data Logging and CSV Export

During processing, maintain an in-memory list `interaction_records`. Each time an interaction is finalized (state machine reaches IDLE after ACTIVE), compute all fields and append a new record.

Logging triggers:

- Append when: state transitions `PENDING_END → IDLE` after `frames_gap > G_GAP`.
- Append when: video ends and any interaction is still in `ACTIVE` or `PENDING_END` state (force-finalize it).

Export:

- At the end of each video (or every 500 interactions for long streams), write all records to a CSV file using Pandas.

- File naming convention: `interaction_report_<video_id>_<timestamp>.csv`.
- Include a header row matching all columns defined in Section 3.1 (CSV Report).

13. Step 8 — Annotated Video Output

For every frame written to the annotated output video:

Always draw:

- Bounding boxes for all tracked persons (color: green, line width: 1).
- Track ID label above each box.

When state is ACTIVE or PENDING_END for a pair:

- Attacker bounding box: **red**, line width: 3.
- Victim bounding box: **yellow**, line width: 3.
- Line between attacker center and victim center: **white**, line width: 2.
- Text overlay at top-left of victim box:
  - `INT#<interaction_id> dur=<X.Xs>` during active phase.
  - `INT#<interaction_id> [SUSPICIOUS] dur=<X.Xs>` in red on finalization.
  - `INT#<interaction_id> [NORMAL] dur=<X.Xs>` in green on finalization.

Optional enhancement:

- Lightly darken (multiply pixel values by 0.5) all regions outside the attacker and victim bounding boxes during ACTIVE state to focus attention on the interaction pair.

Output file:

- File naming convention: `annotated_<video_id>.mp4`.
- Codec: mp4v.
- Frame rate: same as input video fps.

14. Integration with Phase 2 Social Force Model (Gap 3)

14.1 Loading Force Events

When `OPERATING_MODE = "event_driven"`, load `forceevents.json` before processing the video. Each entry contains at minimum:

- `frame` (int): frame number.
- `trackid1`, `trackid2` (int): the two persons involved.
- `forcemagnitude` (float): anomalous force value.
- Optionally: `vx1, vy1, vx2, vy2` (float): velocity components.

Build a lookup structure: `force_by_frame[frame_number]` = list of (trackid1, trackid2, forcemagnitude, velocities).

14.2 Event-Driven State Machine Activation

In event-driven mode:

- Only create a state machine for `(trackid1, trackid2)` if the pair appears in `force_by_frame`.
- Activate the state machine starting from `frame - FORCE_LOOKBACK_FRAMES` to also capture any pre-contact build-up.
- Continue running the state machine for up to `FORCE_LOOKAHEAD_FRAMES` after the last force event for that pair.

Configuration variables:

Variable	Recommended Value	Purpose
`FORCE_LOOKBACK_FRAMES`	75 (≈ 3 s at 25 fps)	Frames before force event to start tracking pair
`FORCE_LOOKAHEAD_FRAMES`	100 (≈ 4 s at 25 fps)	Frames after last force event to keep tracking pair

14.3 Enriching Interaction Records with Force Data

Whether in continuous or event-driven mode:

- For each finalized interaction, search `force_by_frame` for entries involving this `(attacker_id, victim_id)` pair within `[start_frame, end_frame]`.
- Set `max_force_magnitude` = maximum force value found.

- If velocities are available in force events and `VELOCITY_SMOOTHING_FRAMES` smoothing has not yet been applied, use Phase 2 velocities as a fallback to compute `peak_relative_speed_bhs`.

14.4 Continuous Mode Fallback

When `forceevents.json` is not available (standalone testing, early development):

- Set `OPERATING_MODE = "continuous"`.
- Run state machines for all attacker–victim pairs.
- `max_force_magnitude` remains null in CSV output.
- All other functionality is identical.

15. Integration with Immediate Evidence Capture

When a finalized interaction is labeled `"suspicious"`:

Trigger data to pass:

Field	Value
`video_id`	Source video identifier
`interaction_id`	Interaction record ID
`attacker_id`, `victim_id`	Track IDs
`capture_start_time_sec`	`start_time_sec - PRE_MARGIN_SEC`
`capture_end_time_sec`	`end_time_sec + POST_MARGIN_SEC`
`label`	`"suspicious"`
`ml_prob_suspicious`	Float, or null if ML not used

Configuration variables:

Variable	Recommended Value	Purpose
`PRE_MARGIN_SEC`	5.0	Seconds of footage to capture before interaction start
`POST_MARGIN_SEC`	5.0	Seconds of footage to capture after interaction end

The Evidence Capture module uses its rolling frame buffer to dump a video clip covering [capture_start_time_sec, capture_end_time_sec] and to save key frames and timestamped evidence files.

16. Edge Cases and Robustness Handling

16.1 ID Switches and Tracking Errors

DeepSORT/ByteTrack may occasionally reassign IDs when persons heavily occlude each other or cross paths.

Mitigation strategies:

- The `G_GAP` parameter already handles brief occlusions up to `G_GAP / fps` seconds.
- For advanced robustness: if an attacker ID disappears and a new ID appears within 1–2 seconds at the same spatial location and with similar velocity, treat them as the same person and merge their interaction records.
- Log ID-switch events for debugging using track position and velocity continuity checks.

16.2 Multiple Nearby People

An attacker may simultaneously be close to multiple people.

Handling strategy:

- Maintain independent state machines for all (attacker, victim) pairs.
- Designate the person with the smallest `d_norm` to the attacker as the **primary victim**.
- Still log and classify all pairs in the CSV.
- For Evidence Capture triggers, use the primary victim's interaction record.
- In the annotated video, highlight only the primary pair at full intensity; draw secondary pairs with a dashed or thinner box.

16.3 Long Non-Criminal Conversations

Two friends standing close for > 10 seconds with low relative motion are a common false positive source.

Handling:

- T_LONG_SEC = 10.0 ensures that long-duration, low-speed interactions default to "normal".
- The number of gap events (num_gap_events) feature captures conversational ebb-and-flow (people lean in and out), which is distinct from a snatch.
- The post-contact duration feature captures whether both persons remain in the scene together after the interaction (friends do; attacker-victim pairs often don't).

16.4 Ultra-Short Path Crossings

Two unrelated people passing in opposite directions may be briefly close (< 0.5 seconds).

Handling:

- T_MIN_SEC = 0.20 and K_START = 5 (≈ 0.2 s at 25 fps) ensure that interactions shorter than roughly 5 frames are never classified as suspicious.
- At 25 fps, a person crossing another's path at normal walking speed will remain within D_CLOSE for approximately 2–4 frames — well below K_START.

17. Configuration Reference

All thresholds and parameters should be centralized in a configuration file (e.g., config.yaml or config.py) for easy tuning without modifying core module code.

Variable	Recommended Value	Category
FPS	25.0	Video
OPERATING_MODE	"continuous" or "event_driven"	Mode
YOLO_CONF_THRESHOLD	0.30	Detection
YOLO_IOU_THRESHOLD	0.45	Detection
TRACKER_MAX_AGE	30	Tracking
TRACKER_MIN_HITS	3	Tracking
D_CLOSE	0.70	Interaction
K_START	5	Interaction
G_GAP	8	Interaction
H_OVERLAP_CLOSE	0.30	Distance

IOU_CLOSE	0.10	Distance
VELOCITY_SMOOTHING_FRAMES	4	Motion
V_HIGH_BHS	1.5	Motion
V_NOISE_FLOOR_BHS	0.05	Motion
T_MIN_SEC	0.20	Classification
T_SHORT_SEC	2.50	Classification
T_LONG_SEC	10.0	Classification
UNCERTAIN_DEFAULT	"suspicious"	Classification
ML_PROB_THRESHOLD	0.50	ML Classifier
USE_POSE_DISTANCE	False	Optional features
FORCE_LOOKBACK_FRAMES	75	Phase 2 integration
FORCE_LOOKAHEAD_FRAMES	100	Phase 2 integration
PRE_MARGIN_SEC	5.0	Evidence capture
POST_MARGIN_SEC	5.0	Evidence capture

18. Evaluation and Final Deliverables

18.1 Accuracy Targets

Metric	Target	Level
Accuracy	≥ 90%	Both frame-level and interaction-level
Precision (suspicious)	≥ 88%	Interaction-level
Recall (suspicious)	≥ 90%	Interaction-level
F1 (suspicious)	≥ 89%	Interaction-level
False positive rate	≤ 10%	Interaction-level

18.2 Validation Setup

- Use labeled CCTV datasets with ground truth start/end times and suspicious/normal labels for each interaction.
- Split by video (not by frame or interaction): 70% train, 15% validation, 15% test.
- All videos in each split must be disjoint — no video appears in more than one split.
- Do not apply any artificial ceiling on the number of videos per split.
- Report confusion matrices at both frame and interaction level.
- Report metrics separately for: (a) rule-based classifier only, (b) ML classifier only, (c) hybrid rule + ML.

18.3 Final Deliverables Checklist

Deliverable	Description
Interaction duration module	Complete Python implementation of all steps in this spec
Config file	`config.yaml` or `config.py` with all variables from Section 17
CSV outputs	One CSV per test video showing all logged interaction records
Annotated videos	Annotated `.mp4` files for ≥ 20 test clips
Phase 2 integration hook	Function to load `forceevents.json` and run event-driven mode
Evidence Capture trigger hook	Function to emit trigger events to the Immediate Evidence Capture module
ML classifier (optional)	Trained model file with training and evaluation script
Evaluation report	Accuracy, precision, recall, F1 at both frame and interaction level, confirming ≥ 90% accuracy

This specification is intentionally code-free. It provides complete architecture, variable names, recommended values, data flows, edge case handling, and accuracy targets so that a code generation tool (e.g., Codex) can directly produce a complete, production-ready implementation without requiring further clarification.